Cagayan De Oro College PHINMA Education Network

STUDENT GRADE MONITORING SYSTEM

Introduction To Computing Project
Presented to the
Faculty of the College of Information Technology
Cagayan de Oro College
Cagayan de Oro City

In Partial Fulfillment of the
Requirements for the Degree
Bachelor of Science in Information Technology

NAME OF RESEARCHER

BABAYEN-ON VINCENT P.
AJOSAN ROBERT REV T.
GAANE, IDEL KATRINA
MANA VHODKA ROBI
LEQUIT, JOHN DAVE
UROT XYRUS A.

OCTOBER, 2026

1.1 Introduction

Grade recording and monitoring are important in helping students understand their academic performance. A digital grade-recording system can organize grades by subject and make academic information easier to check and review. Research also shows that feedback and monitoring can support students' academic achievement and help them identify areas that need improvement (Newman et al., 2021; Dignath et al., 2023).

For this project, the assumed setting is a college environment where students record and check their grades from different subjects ann7d activities. Based on the project proposal, three problems are identified: (1) difficulty tracking grades from multiple subjects, (2) difficulty organizing grade records, and (3) difficulty identifying areas that need improvement. The proposal states that grades may be recorded in notebooks, spreadsheets, or announced verbally, which can cause confusion or missed updates.

The proponents developed the Student Grade Recording System using a Python console-based application to help users record, organize, and monitor academic grades. The primary users are students who can view and monitor their grades, while teachers may input and update academic records. The system includes grade entry, subject and activity records, grade viewing, automatic average calculation, and performance monitoring. These functions address the three problems by organizing grades in one system, making records easier to track, and helping users identify subjects that may need improvement.

1.2 Statement of the Problem

General Problem

Students have difficulty tracking and organizing their grades from different subjects and activities. This can make it difficult for them to know their academic standing and identify areas that need improvement.

Specific Problems

1. Difficulty tracking grades – Students may have trouble keeping track of grades from multiple subjects, which can cause missed or outdated information.
2. Difficulty organizing grade records – Grades recorded in different places can become confusing and difficult to manage.
3. Difficulty identifying areas for improvement – Students may struggle to determine which subjects need more attention because their grades are not organized and summarized.

1.3 Objectives of the Study

General Objective

To develop the Student Grade Recording System, a Python console-based application that enables students to record, organize, and monitor their grades. Specific Objectives

1. To record and store grades from different subjects and activities to address the difficulty of tracking grades.
2. To organize and display grade records in one system to make grades easier to manage. 3. To calculate and monitor average grades to help students identify subjects that need improvement.

1.4 Scope

The system will focus on recording, organizing, and monitoring student grades. It will include subject listings, grade entries, and performance summaries.

- Students can enter and save their grades for different subjects and activities.
- Students can view their recorded grades and calculated averages to monitor their academic performance.

1.5 Limitations

The system will not handle official school grading systems, enrollment processes, or academic record validation. It is only intended for monitoring and tracking academic performance.

1.6 System Requirements

Hardware Requirements

Hardware Component	**Purpose**
Laptop or Desktop Computer	Used for developing, running, and managing the Grade Monitoring System application
Smartphone	Used for convenient access to grade records anytime and anywhere.
Keyboard and Mouse	Used for inputting data and navigating the system efficiently.

Software Requirements

Software / Application	**Purpose**
Operating system	Provides the main platform for running the application and development tools.
Web browser	used for accessing and testing the system if it is web-based
Programming Language	Used to develop the logic and functionality of the Student Expense Tracker.
Code editor	Used for writing, editing, and managing the source code

1.8 Code for STUDENT GRADE MONITORING SYSTEM

Session records and subject permissions

```
Def grade_key(student_id, subject, semester, period):
    Return "|".join((student_id, subject, semester, period))

Def get_grade(data, student_id, subject, semester, period):
    Return data["grades"].get(grade_key(student_id, subject, semester, period))

Def subject_records(data, student_id, subject, semester):
    Return [record for period in PERIODS
        For record in [get_grade(data, student_id, subject, semester, period)] if record]

Def semester_final(data, student_id, subject, semester, preview_period=None,
preview_grade=None):
    """Use the supplied subject rule and keep the two semesters separate."""
    If subject in MAJOR_SUBJECTS:
        If preview_period == "P3":
            Return preview_grade
        Record = get_grade(data, student_id, subject, semester, "P3")
```

```
        Return record[“grade”] if record else None

    Grades = []

    For period in PERIODS:

        Record = get_grade(data, student_id, subject, semester, period)

        Grade = preview_grade if period == preview_period else record[“grade”] if record
else None

        If grade is None:

            Return None

        Grades.append(grade)

    Weighted = sum(Decimal(str(grade)) * weight for grade, weight in zip(grades,
MINOR_PERIOD_WEIGHTS))

    Return float(weighted.quantize(Decimal(“0.01”), rounding=ROUND_HALF_UP))



Def final_text(value):

    Return “Pending” if value is None else “{:.2f}”.format(value)



Def grading_rule(subject):

    If subject in MAJOR_SUBJECTS:

        Return “Major: P3 is the cumulative final.”

    Return “Minor: P1 33% + P2 33% + P3 34%.”
```

```
Def normalize_mastery(mastery):

    “””Normalize the two Mastery components to a percentage out of 100.”””

    If not isinstance(mastery, dict):

        Raise ValueError(“Enter Wrap-Up Quiz and Final Project percentages for the P3 Mastery result.”)

    Scores = []

    For component in (“wrap_up_quiz”, “final_project”):

        Value = mastery.get(component)

        If type(value) not in (int, float) or not math.isfinite(value) or not MIN_GRADE <= value <= MAX_GRADE:

            Raise ValueError(“Each Mastery component must be a percentage from 0 to 100.”)

        Scores.append(Decimal(str(value)))

    Score = (scores[0] * Decimal(“0.15”) + scores[1] * Decimal(“0.30”)) / Decimal(“0.45”)

    Return {“wrap_up_quiz”: float(scores[0]), “final_project”: float(scores[1]),

            “score”: float(score.quantize(Decimal(“0.01”), rounding=ROUND_HALF_UP))}


Def semester_result(data, student_id, subject, semester, preview_period=None,

                    Preview_grade=None, preview_mastery=None):
```

```
    Score = semester_final(data, student_id, subject, semester, preview_period,
preview_grade)

    Result = {“score”: score, “grade”: “Pending”, “status”: “Pending”, “mastery”: None}

    If score is None:

        Return result

    If subject not in MAJOR_SUBJECTS:

        For minimum, grade in MINOR_GRADE_BANDS:

            If score >= minimum:

                Result.update(grade=grade, status=”Passed” if score >= 50 else “No Credit”)

                Return result

    Record = get_grade(data, student_id, subject, semester, “P3”)

    Mastery = preview_mastery if preview_period == “P3” else record.get(“mastery”) if
record else None

    Result.update(grade=”{:.2f}%”.format(score), mastery=mastery)

    If mastery is not None:

        Result[“status”] = “Passed” if score >= 80 and mastery[“score”] >= 80 else “Failed”

    Return result


Def show_final_result(result, subject):

    If result[“score”] is None:
```

```
        Print("Final: Pending")

        Return

    If subject in MAJOR_SUBJECTS:

        Print("Final: {}% | {}".format(final_text(result["score"]), result["status"]))

    Else:

        Print("Final: {}% | Grade: {} | {}".format(final_text(result["score"]),
result["grade"], result["status"]))

    If subject in MAJOR_SUBJECTS:

        If result["mastery"] is not None:

            Mastery = result["mastery"]

            Print("Mastery: {}% | Quiz: {}% | Project:
{}%".format(final_text(mastery["score"]),

                Format_grade(mastery["wrap_up_quiz"]),
format_grade(mastery["final_project"])))



Def can_edit_subject(faculty_id, subject):

    Account = FACULTY.get(faculty_id)

    Return account is not None and account["subject"] == subject



Def subject_owner(subject):

    Return next(faculty_id for faculty_id, account in FACULTY.items()
```

```
        If account[“subject”] == subject)


Def post_grade(data, faculty_id, student_id, subject, semester, period, grade, note,
mastery=None):
    “””Check subject permissions again at the point where a grade changes.”””
    If not can_edit_subject(faculty_id, subject):
        Raise PermissionError(“View only: you can record grades only for your assigned
subject.”)
    If student_id not in data[“students”]:
        Raise ValueError(“Student ID not found.”)
    If not data[“students”][student_id][“full_name”]:
        Raise ValueError(“Student profile must include a full name.”)
    If student_id not in data[“class_lists”][faculty_id]:
        Raise PermissionError(“Add this student to your class list first.”)
    If semester not in SEMESTERS or period not in PERIODS:
        Raise ValueError(“Select a valid semester and period.”)
    Note = validate_note(note)
    If type(grade) not in (int, float) or not math.isfinite(grade) or not MIN_GRADE <=
grade <= MAX_GRADE:
        Raise ValueError(“Grade must be a number from 0 to 100.”)
```

```
    If subject in MAJOR_SUBJECTS and period == “P3”:

        Mastery = normalize_mastery(mastery)

    Elif mastery is not None:

        Raise ValueError(“Mastery components apply only to the P3 final for major
subjects.”)

    Old = get_grade(data, student_id, subject, semester, period)

    If old and (old[“grade”], old[“note”], old.get(“mastery”)) == (grade, note, mastery):

        Return False

    Data[“grades”][grade_key(student_id, subject, semester, period)] = {

        “student_id”: student_id, “subject”: subject, “semester”: semester, “period”: period,

        “grade”: grade, “note”: note,

        “faculty_id”: faculty_id, “mastery”: mastery, “updated_at”: now(),

    }

    Score_change = “{} to {}”.format(format_grade(old[“grade”]), format_grade(grade))
if old else format_grade(grade)

    Message = “Score: {}%.”.format(score_change)

    Result = semester_result(data, student_id, subject, semester)

    If result[“score”] is not None:

        Message += “ Final: {}% | Grade: {} | {}.”.format(final_text(result[“score”]),
result[“grade”], result[“status”])

    If result[“mastery”] is not None:
```

```
        Message += " Mastery: {}%.".format(final_text(result["mastery"]["score"]))
    If note:
        Message += " Note: " + note
    Add_event(data, student_id, "{} | {} | {} Grade {}".format(
        SEMESTERS[semester], subject, period, "Updated" if old else "Posted"), message)
    Return True


Def add_event(data, student_id, title, message):
    Data["events"].append({
        "id": max((event["id"] for event in data["events"]), default=0) + 1,
        "student_id": student_id, "title": title, "message": message,
        "created_at": now(), "seen": False,
    })



# Login
Def grade_board(data, student_id, semester, faculty_id=None):
    Print(SEMESTERS[semester] + " Grade Board\n")
    Subjects = list(SUBJECTS)
```

```
    Rows = []

    For number, subject in enumerate(subjects, 1):

        Grades = [get_grade(data, student_id, subject, semester, period) for period in
PERIODS]

        Result = semester_result(data, student_id, subject, semester)

        Status = result[“status”]

        If faculty_id is not None:

            Status = “Can Edit” if can_edit_subject(faculty_id, subject) else “View Only”

        Rows.append([number, subject, SUBJECT_LABELS[subject]] +

                    [format_grade(record[“grade”]) if record else “N/A” for record in grades] +

                    [final_text(result[“score”]), result[“grade”], status])

    Table([“No.”, “Code”, “Subject”, “P1”, “P2”, “P3”, “Final %”, “Grade”, “Access” if
faculty_id else “Result”],

          Rows, [3, 7, 20, 5, 5, 5, 7, 7, 11])

    Print(“\nN/A = Not posted | Pending = Final not ready”)

    Return subjects


Def subject_details(data, student_id, subject, semester):

    Paragraph(subject + “ – “ + SUBJECTS[subject])

    Print(SEMESTERS[semester])
```

```
    Records = subject_records(data, student_id, subject, semester)

    Show_final_result(semester_result(data, student_id, subject, semester), subject)

    Missing = [period for period in PERIODS if not get_grade(data, student_id, subject, semester, period)]

    If missing:

        Print(“Not posted: “ + “, “.join(missing))

    If not records:

        Return

    Print()

    Table([“Period”, “Score”, “Note”],

          [[record[“period”], format_grade(record[“grade”]) + “%”, record[“note”]]

           For record in records], [6, 7, 60])

    If len(records) >= 2:

        Previous, latest = records[-2:]

        Change = round(latest[“grade”] – previous[“grade”], 2)

        Progress = “No change” if change == 0 else (“+” if change > 0 else “-“) + format_grade(abs(change)) + “ points”

        Print(“\n{} to {}: {}”.format(previous[“period”], latest[“period”], progress))
```

References:

Newman, M., Kwan, I., Schucan Bird, K., & Hoo, H.-T. (2021). The impact of feedback on student attainment: A systematic review. Education Endowment Foundation.

Koenka, A. C., Linnenbrink-Garcia, L., Moshontz, H., Atkinson, K. M., Sanchez, C. E., & Cooper, H. (2021). A meta-analysis on the impact of grades and comments on academic motivation and achievement: A case for written feedback. Educational Psychology, 41(7), 922–947.

Arora, P., & Wright, N. (2022). Grade reporting and student performance. Education Economics, 30(4), 356–363.

Seldow, A. L. (2010). On-demand grades: The effect of online grade book access on student mastery and performance goal orientations, grade orientation, academic selfefficacy, and grades [Doctoral dissertation, Harvard University]. ProQuest.

Python Software Foundation. (2026). Python 3.14.7 documentation. Python.org.

Python Software Foundation. (2026). The Python tutorial: Data structures. Python.org.

Newman et al. (2021) and Koenka et al. (2021)

Cagayan De Oro College PHINMA Education Network

Python Software Foundation (2026)